

Explicação do Código Java: ManualSemaphore

1. Implementa um semáforo manual para controlar acesso concorrente de threads a um recurso limitado.
2. O campo `permits` guarda quantas permissões estão disponíveis; inicia com o valor passado ao construtor.
3. `acquire()` é `synchronized`: se `permits == 0`, a thread entra em espera com `wait()` até alguém liberar vaga.
4. Quando há vaga, `acquire()` decrementa `permits` e permite a entrada da thread na seção controlada.
5. `release()` também é `synchronized`: incrementa `permits` e acorda uma thread bloqueada via `notify()`.
6. No main, 4 threads competem por apenas 2 permissões, então no máximo 2 executam simultaneamente.
7. Conceitos usados: multithreading, exclusão mútua, monitor intrínseco de objetos, `wait/notify` e lambdas.
8. Complexidade por operação: $O(1)$; uso de memória $O(1)$, desconsiderando overhead das threads.
9. Decisão importante: uso de `while` evita avanço indevido após wakeups espúrios ou disputa entre threads.
10. Limitação: `notify()` pode ser menos justo/eficiente que `notifyAll()`; `java.util.concurrent.Semaphore` é mais robusto.